

PULSE

Social Media Trend and Sentiment Analysis Using Big Data

Mini Project Report — Big Data Analysis (CSC702)

B.E. CMPN, Semester VII · St. Francis Institute of Technology, Mumbai

Item	Detail
Application name	PULSE — Social Intelligence (web application)
Dataset	TweetEval sentiment test split: 12,284 labelled English tweets (public dataset, bundled locally)
Lab experiments implemented	Exp. 4 (MapReduce word count), Exp. 6 (Bloom filter), Exp. 7 (Social network analysis)
Technology	TypeScript, React, TanStack Start/Router, Vite, Tailwind CSS, Recharts, Papa Parse, Vitest
How to run	npm install, then npm run dev, then open http://localhost:8080

1. Introduction and objective

Social media produces a continuous, very large stream of short text. Understanding what people feel and which topics are rising is a classic big data problem: the data is high-volume, high-velocity and unstructured. This project, **PULSE**, is a web dashboard that analyses a real labelled social media dataset to show public opinion (positive, neutral, negative), trending hashtags and sentiment over time.

Because the subject is Big Data Analysis, the project goes beyond a dashboard. It implements three experiments from the lab manual — **MapReduce**, a **Bloom filter** and **social network analysis** — directly on the project dataset, so each big-data concept is demonstrated on real data and not on toy examples.

Objectives

- Analyse a real labelled social media dataset and present sentiment and trend insights.
- Present the results in a modern, animated, multi-page web interface.
- Apply at least three lab-manual concepts (MapReduce, stream/probabilistic algorithms, graph analytics) to the same data.
- Be honest about data provenance: the app replays a public dataset and does not claim live access to any platform.

2. Dataset

The dataset is the public **TweetEval** sentiment benchmark (test split). It is stored in the project as `src/data/tweets.json`, so the results are reproducible and need no platform credentials or API keys.

Property	Value
Total posts	12,284
Neutral	5,937 (48.3%)
Negative	3,972 (32.3%)
Positive	2,375 (19.3%)
Unique hashtags	3,954 (as counted by the app)
Fields	id, text, sentiment (positive / neutral / negative)

The sentiment labels come from the dataset and were not predicted by this project. All statistics shown in the dashboard (counts, percentages, hashtag tallies) are derived from these preserved labels. A user can also import their own CSV (columns *text* and *sentiment*) in the Dataset Lab and export the data.

3. System design

3.1 Architecture

PULSE is a client-side analytics application. The dataset is bundled with the app and every computation runs in the browser; there is no database or back-end service. The architecture has three layers:

- **Data layer** — `src/data/tweets.json` and `src/data/analysis.ts` (loads posts, counts sentiment, tallies hashtags, builds the 24-batch time series).

- **Algorithm layer** — `src/lib/bigdata.ts` (pure functions for word count, Bloom filter and hashtag network; no UI code, covered by unit tests).
- **Presentation layer** — React components: `dashboard.tsx` (main pages), `bigdata-lab.tsx` (experiment pages), `shell.tsx` (sidebar, top bar, routing shell).

3.2 Pages

Route	Page	What it shows
/	Overview	Key metrics, sentiment-over-time area chart, sentiment donut, top trending topics, mini live feed
/trends	Trending topics	Searchable hashtag cards with counts and dominant sentiment; click for a breakdown
/sentiment	Sentiment analysis	Charts plus a filterable list of labelled opinions
/live	Live stream	Animated replay of dataset posts, one every 4.2 s, with pause and filter
/datasets	Dataset lab	Browse, filter, import a CSV and export data
/sources	Data sources	Source status: only the Twitter benchmark dataset is available
/mapreduce	MapReduce (Exp. 4)	Interactive map, shuffle and reduce word count
/bloom	Bloom filter (Exp. 6)	Lab-style Bloom filter plus large-scale test
/network	Network analysis (Exp. 7)	Hashtag graph, degree histogram, communities

3.3 Design and branding

The interface uses a dark cybernetic theme with a lime accent, glitch-style headings and animated charts. The application name is **PULSE**; its logo is a heartbeat line with corner brackets (the same logo is used as the browser tab icon). Navigation is a persistent sidebar with a separate *Big Data Lab* section for the experiments.

3.4 Honest labelling of data

The "Live stream" page is a **historical dataset replay**: it cycles through the stored tweets to simulate a stream. The badge "DATASET REPLAY" is shown on every page. Reddit, Instagram, YouTube and TikTok were removed from the interface because they require authorised API credentials that the project does not have; showing them as connected would be misleading. Only the Twitter benchmark dataset is listed.

4. Experiment 4 — Word count using MapReduce

4.1 Theory

MapReduce is a programming model for processing large data sets in parallel on a cluster. The input is divided into splits. Each **mapper** processes one split and emits intermediate key-value pairs. The framework then **shuffles and sorts** the pairs so that all values for one key reach the same **reducer**, which combines them into the final result. An optional **combiner** pre-aggregates pairs locally on each mapper to reduce network traffic.

4.2 Implementation in PULSE

- The 12,284 posts are divided into N equal input splits (N = 1 to 12, set by a slider = number of mappers).
- **Map:** each mapper tokenises its posts (lower-case words of 3+ letters, common stop-words removed, hashtags kept) and emits (word, 1).
- **Shuffle:** pairs are grouped by word, giving (word, [1, 1, 2, ...]).
- **Reduce:** each group is summed to give (word, total); the top 15 words are displayed as bars.
- **Combiner:** a checkbox enables local pre-aggregation inside each mapper.

```
map(post)    -> for each word w: emit (w, 1)
combine()    -> per mapper: (w, [1,1,1]) => (w, 3)      [optional]
shuffle()    -> group all pairs by key w
reduce(w,V)  -> emit (w, sum(V))
```

4.3 Results (4 mappers)

Measure	Without combiner	With combiner
Map output pairs	117,212	117,212
Pairs shuffled to reducers	117,212	36,762 (about 69% less)
Unique words (reducer keys)	21,504	21,504
Most frequent words	trump (940), has, his, via, now, how, people, more	identical

Key observations: (1) the final word counts are identical with or without a combiner and for any number of mappers — this is verified by an automated test; (2) the combiner cuts the data moved over the "network" by about two thirds, which is why it matters on a real cluster.

4.4 Limitations

This is a single-machine simulation of the MapReduce flow running in the browser. It illustrates the algorithm exactly but does not use Hadoop, HDFS or Java. The lab manual asks for a Java program (Main, Mapper and Reducer classes) run on Cloudera; that part is done separately in the lab.

5. Experiment 6 — Bloom filter

5.1 Theory

A Bloom filter is a compact, probabilistic data structure that answers "is this element in the set?". It is an array of m bits, initially 0, and k hash functions. To **insert** an element, set the k bits given by its hashes to

1. To **query**, check those k bits: if any bit is 0 the element is **definitely not** present; if all are 1 it is **probably** present. There are no false negatives, but false positives are possible. It is used for stream filtering, web caches, spell checkers, database lookups and spam filtering, because it uses very little memory.

False-positive probability after inserting n elements: $(1 - e^{-kn/m})^k$.

5.2 Implementation in PULSE

Part A — as specified in the lab manual. Filter size $m = 11$ bits (editable). Each word is converted to the sum of its ASCII codes e . Two hash functions are used: $h1 = (\text{roll no.} \times e) \bmod 11$ and $h2 = ((2 \times \text{roll no.} - 1) \times e) \bmod 11$. The user enters 10 words to insert and 5 words to test. The page draws the bit array and prints a table with each test word, its ASCII sum, the bit positions it hashes to and the verdict: *True positive*, *False positive* or *Not present*.

Part B — big-data scale. All 3,954 dataset hashtags are inserted into one large filter (size adjustable from 2,048 to 65,536 bits) and tested against 16,464 ordinary words that are not hashtags, so every "present" answer is a false positive. This shows how filter size controls accuracy.

5.3 Worked example (roll no. 7, $m = 11$, $k = 2$)

Inserted: python, spark, hadoop, hive, bigdata, mongodb, cloud, data, tweet, stream. Resulting bit array (positions 0-10): 0 1 1 1 1 1 1 1 0 1 1 (9 of 11 bits set).

Test word	ASCII sum	Bits	Result
spark	545	9, 1	True positive
data	410	10, 6	True positive
java	418	0, 0	Not present
kafka	510	6, 8	Not present
hbase	515	8, 7	Not present

With only 11 bits the filter is already 82% full, so false positives are very likely for new words — a good demonstration of why m must be large relative to the number of elements. (If a tested word happens to light all its bits without having been inserted, the page marks it *False positive*.)

5.4 Large-scale results (3,954 hashtags, $k = 3$)

Filter size m	Memory	Measured false positives	Theoretical rate
4,096 bits	0.5 KB	62.6%	84.3%
16,384 bits	2 KB	8.3%	13.7%
65,536 bits	8 KB	0.34%	0.45%

The measured rates follow the theoretical curve closely (they fall by the same large factor each time the filter is made four times bigger). A Bloom filter holding all hashtags in 8 KB with under 1% false positives is far smaller than storing the strings themselves.

5.5 Important finding: the ASCII-sum hash is weak

While testing at scale we found that the lab-manual hash functions fail on a large vocabulary: at the same sizes they produce roughly **93% false positives** even with 65,536 bits. The reason is that an ASCII sum

only takes about a thousand distinct values (and anagrams such as "listen" and "silent" share the same sum), so thousands of different words collide before any modulus is applied. Part B therefore uses a proper hash (two FNV-1a hashes combined by double hashing); the page also displays the ASCII-sum result for comparison. This is a useful point to discuss: **the quality of the hash function matters as much as the filter size.**

6. Experiment 7 — Social network analysis

6.1 Theory

Social network analysis (SNA) studies relationships as a graph: **nodes** are entities and **edges** are connections. The **degree** of a node is the number of connections it has; nodes with a very high degree are **hubs**. **Community detection** finds groups of nodes that are more densely connected to each other than to the rest. SNA is used to find influencers, track information spread, detect communities and recommend content.

6.2 Implementation in PULSE

The lab manual uses R (igraph) with the steps: load library, get data, create network, histogram of node degree, network diagram, highlighting degrees and layouts, community detection. PULSE performs the same steps in the browser on the project dataset:

- **Get data:** all 12,284 posts; 2,633 of them contain two or more hashtags.
- **Create network:** each hashtag is a node; two hashtags are joined by an edge whenever they appear in the same post. Edge weight = number of posts they share.
- **Histogram of node degree:** a bar chart of how many hashtags have each degree (whole network, 75 distinct degree values).
- **Network diagram:** an SVG graph of the highest-degree hashtags (10-80, adjustable).
- **Highlighting degrees and layouts:** node size is proportional to degree; a switch toggles a circular layout and a force-directed layout; clicking a node highlights it and its neighbours and dims the rest.
- **Community detection:** label propagation — every node starts with its own label and repeatedly adopts the label with the greatest (degree-normalised, weighted) support among its neighbours until stable. Communities are shown as colours.

6.3 Results (top 40 hashtags)

Measure	Value
Posts with 2+ hashtags	2,633
Nodes / edges shown	40 / 147
Communities found	7
Largest hubs (degree)	#trump (20), #blackfriday (19), #isis (18), #syria (15), #hezbollah (13)
Degree distribution	Heavy-tailed: 611 hashtags with degree 1, 570 with degree 2, 572 with degree 3, falling steadily towards a few large hubs

The distribution shows the typical pattern of real networks: very many low-degree nodes and a small number of hubs. Colours on the diagram mark the detected communities, which can be inspected on the page.

6.4 Limitations

The graph is simplified: the diagram shows only the top-degree hashtags so that it stays readable, and label propagation is a fast heuristic rather than an optimal clustering. R and igraph were not used; the lab manual R steps are done separately.

7. Mapping to the syllabus

Experiment	Lab outcome	Big-data concept shown
4 — MapReduce word count	CSL7012.2 Implement MapReduce algorithms on unstructured data	Parallel split-map-shuffle-reduce, combiners, scalability
6 — Bloom filter	CSL7012.4 Implement data-stream algorithms	Probabilistic membership, space/accuracy trade-off, false positives
7 — Social network analysis	CSL7012.6 Analyse social network graphs with visualisation	Graph construction, degree distribution, hubs, communities

The dashboard itself addresses course outcome CSC702.5 (big data analytics in social media applications): sentiment analysis, trend detection and replayed streaming of a large text corpus.

8. Testing and verification

- Automated tests (Vitest) check: word-count results are identical for any mapper count and with or without a combiner; the combiner reduces shuffled pairs; the Bloom filter has no false negatives and ASCII sums are correct; the hashtag network builds a graph with more than one community; and the app routes resolve.
- The TypeScript compiler reports no errors, and all pages return HTTP 200 on the development server.
- Numbers quoted in this report were produced by running the project code on the dataset.

9. Limitations and future work

- **No live data:** the stream is a replay of a static public dataset. Real live data would need authorised platform APIs and credentials; that is why other platforms were removed from the UI.
- **Single-node simulation:** the experiments illustrate the algorithms in the browser; they are not run on a real Hadoop/Spark cluster.
- **Sentiment labels are given:** the app does not train a model. A natural extension is PySpark MLlib classification (lab Experiment 2), e.g. Naive Bayes or logistic regression, evaluated with accuracy and F1.
- **Future work:** a Spark pipeline for larger corpora, real-time ingestion with Kafka, and storing results in MongoDB (Experiment 3).

10. How to run the project

```
cd cyber-pulse-analytica
npm install
npm run dev      # open http://localhost:8080
npm test         # run the automated tests
```

11. Likely questions from the teacher (with short answers)

Question	Answer
Where is the big data?	12,284 posts with 117,212 word occurrences and 3,954 hashtags; more importantly the algorithms used (MapReduce, Bloom filter, graph analytics) are the techniques that scale to far larger data.
Is the data live?	No. It is a replay of the public TweetEval dataset and is labelled "Dataset replay" throughout. Live APIs need credentials the project does not have.
Who labelled the sentiment?	The dataset authors (TweetEval). The project does not classify sentiment itself.
Why does a combiner help?	It sums counts locally on each mapper, so fewer pairs cross the network: 117,212 pairs became 36,762 here (about 69% fewer) with identical results.
Can a Bloom filter give a wrong answer?	Yes, false positives (says present when it is not). Never false negatives. The false-positive rate falls as the filter gets bigger.
Why did the ASCII-sum hash perform badly at scale?	Only about a thousand distinct sums exist and anagrams share a sum, so many words collide; a good hash (FNV-1a) spreads words evenly.
What is a hub and what is a community?	A hub is a node with very high degree (e.g. #trump, degree 20). A community is a group of nodes more densely connected to each other than to the rest.
Which algorithm finds communities?	Label propagation: nodes repeatedly adopt the most common label among their neighbours until labels stop changing.
Is this really MapReduce?	It follows the same logic (split, map, shuffle, reduce, combine) but runs on one machine in the browser. The Hadoop version is done in Cloudera in the lab.